

Claim Assistant — Technical Stack

1. Confirmed Stack

Layer	Choice	Alternatives Considered	Why I Picked This
Frontend	React	None evaluated	Claims Application UI used by the Claim Processor: submit claims, view/answer agent questions, review decisions.
Agents / backend	Python	None evaluated	Implements the ReAct loop, orchestrator, and tool layer.
LLM	GPT API (platform.openai.com)	None evaluated	Powers the Think step of the reasoning loop (proposes actions); the harness executes them.
Retrieval / RAG	Pinecone	None evaluated.	Vector store for the policy & regulation semantic memory corpus.
Orchestration	LangGraph	None evaluated (typical alternatives here would be a heavier durable-workflow engine like Temporal).	Drives the ReAct (Think/Act/Observe) loop and the Adjudication Orchestrator; its checkpointing can back the "survive pauses, resume on ask_human response" requirement.
Checkpointing	LangGraph PostgresSaver	In-memory checkpointing (MemorySaver) — rejected, doesn't survive a process restart, which breaks the "resume hours/days later" requirement. SqliteSaver — rejected once the Database decision moved to Postgres, to	Persists graph state so a paused run (waiting on ask_human) can be rehydrated later. Backed by the same Postgres instance as the rest of the app.

Layer	Choice	Alternatives Considered	Why I Picked This
		avoid running two DB technologies for no benefit.	
Database	Postgres	SQLite (interim, migrate later) — reconsidered and rejected: deployment already requires a cloud VM, and most hosts (Fly.io, Railway, etc.) attach managed Postgres about as easily as a persistent volume for SQLite, so provisioning Postgres from the start costs about the same as SQLite + a later migration, without ever hitting SQLite's write-concurrency ceiling (database is locked under concurrent Check Ledger writes) or needing WAL-mode/busy-timeout workarounds.	Claims DB / Check Ledger / Audit Trail, from the start. Proper row-level locking and MVCC handle concurrent writes from the Orchestrator + multiple in-flight agent runs cleanly — no interim data store, no migration step later
API layer	FastAPI	None evaluated in this session (Flask and Django REST are the usual alternatives; FastAPI's native async support and auto-generated OpenAPI docs are why it's typically preferred alongside an async LangGraph backend).	Sits between the React frontend and the LangGraph backend; exposes claim submission, status, and ask_human question/answer endpoints.
Background execution	FastAPI BackgroundTasks	A task queue (Celery/RQ) — rejected, adds a	Runs the LangGraph agent loop for a submitted claim without blocking the FastAPI

Layer	Choice	Alternatives Considered	Why I Picked This
		broker (Redis/etc.) and worker process not justified at capstone/single-VM scale. Running the agent loop synchronously in the request handler — rejected, blocks the HTTP response for the full multi-minute claim run and is incompatible with the polling-based ask_human design, which assumes the run is already progressing independently of any single request.	request that started it, so the frontend can immediately start polling for status/questions while the run executes. No new infra beyond what's already in the stack.
Embeddings	OpenAI text-embedding-3-small	text-embedding-3-large — deferred; higher quality/cost, only worth it if retrieval quality is measurably lacking.	Same vendor as the LLM — one API key, one billing relationship, no extra integration. Must stay identical between ingestion and query time.
Reranker	LLM-based rerank (GPT)	Cohere Rerank — rejected, adds a new vendor/API key/cost just for this one step. Local cross-encoder — rejected, adds model-serving infra to build and maintain.	Prompt GPT to score/reorder the top-20 Pinecone candidates down to top-3. No new API key/vendor; at k=20 the extra latency/cost per query is negligible. Revisit a dedicated reranker only if rerank quality becomes a bottleneck.
Document ingestion	unstructured + custom clause chunker	A simpler custom parser keyed off consistent clause numbering — rejected since source docs aren't finalized and may not have consistent structure. A	Source policy docs aren't finalized (mix of Word/PDF, structure unknown), so unstructured handles parsing/table-extraction across formats, feeding a custom chunker that splits on clause boundaries per

Layer	Choice	Alternatives Considered	Why I Picked This
		PDF-specific heavy-parsing approach — rejected for the same reason (format not locked in).	requirements.md §9 rather than fixed size.
Observability	LangSmith	LangFuse — rejected, open-source alternative but adds self-hosting overhead not justified for a capstone. Custom structured logging straight into the Audit Trail — rejected as the primary dev tool since it would conflate a compliance-facing store with a dev-debugging concern.	Native LangGraph tracing for development/debugging (step-by-step Think/Act/Observe visibility, latency, token cost). Distinct from the Audit Trail: LangSmith is a dev-facing tool, the Audit Trail (Postgres) remains the compliance system of record — don't conflate the two. Traces will include the full claim trajectory; fine on this project's synthetic data, but revisit before ever pointing this at real customer data.
ask_human channel	Polling	WebSockets — rejected, connection-management complexity (reconnects, scaling) not justified by a human-paced workflow. SSE — rejected as an unnecessary middle ground when polling is simple enough and sufficient.	React polls a FastAPI status endpoint for pending questions/answers. Simplest to build, no connection-management overhead; fine when sub-second latency isn't needed.
Synthetic data	LLM-generated	Hand-authored fixtures — rejected; more deterministic/auditable but more manual effort to produce. Script-generated (Faker/random) — rejected; more	GPT generates the simulated transaction history system logs, and account red-flag records for the 10 test claims. Review each generated record against its claim's intended expected outcome so the synthetic evidence doesn't

Layer	Choice	Alternatives Considered	Why I Picked This
		volume/variety but harder to hand-verify a claim's evidence matches its intended expected outcome.	accidentally contradict the test's expected decision.
Test harness	Notebook-based	pytest + fixtures — rejected; more rigorous and CI-ready, but more ceremony than needed for a capstone eval/demo. Custom eval script — rejected; a viable middle ground, but a notebook is better suited to interactive trace inspection for the writeup.	The 10-claim evaluation set (requirements.md §12) is run and inspected in a Jupyter notebook — each claim's agent trace and decision reviewed interactively. Fits a capstone demo/writeup; not wired into CI.
Deployment	Self-managed Ubuntu VM (existing server)	Local only — rejected; no reachable URL for a demo. A managed PaaS (Fly.io/Railway) — not needed; a suitable Ubuntu server is already available.	FastAPI backend (via systemd + uvicorn), the built React static files (served via Nginx), and Postgres all run on the existing Ubuntu VM, reachable over the server's IP (or a domain if one is pointed at it). Setup is manual (SSH, package installs, systemd unit, Nginx reverse-proxy config) rather than a PaaS's managed build/deploy pipeline — see tracker.md Phase 9 for the concrete steps.
Auth	Shared-password gate	None / single-user — rejected once the app is reachable at a public VM URL, it needs some gate. Basic user accounts — rejected; more build effort than justified for a	One shared password/API key protects the FastAPI endpoints on the public demo VM — keeps the URL from being wide open, without building real per-user accounts. Note: this means the Audit Trail can't attribute an action to a specific processor if more than one

Layer	Choice	Alternatives Considered	Why I Picked This
		capstone that isn't about auth.	person ever uses it — acceptable for a single-user capstone demo, but not a substitute for real per-user auth if this became multi-user.
Secrets management	<code>.env + python-dotenv</code>	Cloud secrets manager — rejected; more production-realistic and slightly safer, but adds setup not justified at capstone/single-VM scale.	OpenAI/Pinecone/LangSmith keys live in a gitignored <code>.env</code> , loaded via <code>python-dotenv</code> ; the same file is copied (never committed) onto the deploy VM.

2. Architecture Components

Derived from `specs/Capstone Claim Project v2.drawio`.

- **Claims Application (Frontend, React)** — entry point for the Claim Processor; reads/writes the Claims DB.
- **Claims DB** — system of record for claim data and final determinations. Postgres.
- **Adjudication Orchestrator** — runs the agent loop for a claim via FastAPI `BackgroundTasks`, so it starts/resumes without blocking the request that triggered it, and must survive pauses (e.g., while waiting on `ask_human`). LangGraph's `PostgresSaver` checkpointer persists graph state between steps so a run can be rehydrated after a pause.
- **ReAct Agent** — the Think/Act/Observe loop, implemented as a LangGraph graph. LLM proposes the next action; the graph/harness is responsible for actually executing it (tool call, human question, or write).
- **Check Ledger** — persistent store of per-check state (PASS / FAIL / UNKNOWN / BLOCKED) for the claim currently being processed; the source of truth the Approve/Deny/Inconclusive decision is derived from. Postgres.
- **Audit Trail** — append-only log of every agent step, tool call, retrieval detail, and final determination + basis. Postgres.
- **Tool Layer**
 - Retrieval tools (claim/dispute history, policy search)
 - Grounding tools (transaction evidence, account/cardholder verification, document extraction with citations)

- Computation tools (date math, duplicate-charge detection, transaction-pattern anomaly scoring)
- `ask_human` (suspends the run; last-resort retrieval)
- `write_determination` (single irreversible write of the final decision)
- **Long-Term Memory**
 - Semantic — policy/regulation corpus, versioned, stored in Pinecone as embeddings.
 - Episodic — entity facts (keyed by account/cardholder), provenance-tagged, so previously verified facts can be reused across claims. Stored as a Postgres table (`episodic_facts` or similar), keyed by entity ID, each row carrying the fact, its source (claim/tool that established it), and a timestamp — a keyed-lookup access pattern, not similarity search, so it belongs in Postgres rather than Pinecone.

Memory-to-store mapping

requirements.md §7 defines short-term vs. long-term memory conceptually; this maps each to an actual store in this stack:

Memory type	What it holds	Backing store
Short-term — agent state	Raw Think/Act/Observe trajectory (messages, tool calls) for the in-progress claim; enables resume after <code>ask_human</code>	LangGraph PostgresSaver checkpointer
Short-term — check state	Structured PASS/FAIL/UNKNOWN/BLOCKED per required check; what the final decision is derived from	Check Ledger (Postgres table)
Long-term — semantic	Policy/regulation corpus, versioned	Pinecone (vector store, similarity search)
Long-term — episodic	Entity facts keyed by account/cardholder, provenance-tagged	<code>episodic_facts</code> (Postgres table, keyed lookup)

All four now live in this stack — no memory type from requirements.md §7 is unassigned.

3. Retrieval Pipeline (RAG)

- Source documents: policy/regulation Word/PDF files, chunked by policy clause (not fixed-size), preserving parent-child structure between a policy and its sub-clauses; tables/decision trees converted to markdown and kept as a single, unsplit chunk.

- Embedding + similarity search via Pinecone.
- Query flow: retrieve top **k ≈ 20** → rerank → apply a relevance-floor threshold → return top **3** (zero results is a valid outcome below the floor).
- Retrieval calls (query, filters, full candidate list, scores) are logged to the audit trail for later audit review, not just the final top-3 used.

4. Claim Taxonomy (Phase 4)

requirements.md §4 says the checks that apply are "determined only after reading the claim" but doesn't enumerate claim types or checks. This section is that enumeration — the concrete mapping the LangGraph agent, synthetic-data fixtures, and (later) the Phase 8 eval set are all built against. Two claim types, matching requirements.md §1's "billing dispute or fraud claims" framing.

Each check is deterministically required for its claim type (looked up by `claim_type`, not LLM-inferred) — the check *ledger* is initialized with all required checks at UNKNOWN when a run starts. What the ReAct loop actually reasons about is *which tool to call next* to resolve each still-UNKNOWN/BLOCKED check, not which checks apply in the first place. This keeps "classification" auditable/deterministic rather than a model judgment call, consistent with requirements.md §13's determinism-of-decisioning requirement.

billing_dispute

Check	Tool category	Resolves by
<code>transaction_exists</code>	Grounding	Disputed transaction is found in the account's transaction history
<code>duplicate_charge_check</code>	Computation	Same amount/merchant charged twice within a short window (only decisive when dispute reason is <code>duplicate_charge</code>)
<code>policy_dispute_window</code>	Retrieval	Retrieve the applicable dispute-filing-window policy; a retrieved, citable policy satisfies the check (see note below — a real day-count comparison against the claim's filed date is deferred)
<code>account_standing</code>	Grounding	Account is not already flagged for dispute-process abuse

fraud

Check	Tool category	Resolves by
account_red_flags	Grounding	Existing fraud/red-flag signals already on the account
transaction_pattern_anomaly	Grounding + Computation	Disputed transaction checked against the account's history for velocity/amount/location anomalies
system_access_log_check	Grounding	Suspicious login/device activity in the access logs around the transaction time
policy_liability_rule	Retrieval	Retrieve the applicable fraud-liability regulation/policy to ground the decision

Synthetic data fixture tables (Postgres)

Backing the Grounding tools above — synthetic, not real customer data (requirements.md §3):

- `transactions` — per-account transaction history (amount, merchant, location, channel, status, timestamp).
- `access_logs` — per-account system/login/device event history.
- `account_profiles` — per-account standing, existing fraud red-flags, dispute history count.

Retrieval tool status

Wired to the real Pinecone index (`claims-policy-corpus`) end-to-end rather than stubbed — since Phase 3 (RAG ingestion) hasn't loaded any policy documents yet, the index currently has 0 vectors, so retrieval calls honestly return zero results. This is the documented, valid "no matching policy found" outcome (requirements.md §9), and means `policy_dispute_window` / `policy_liability_rule` will resolve BLOCKED until Phase 3 lands — no retrieval code changes needed once it does.

Both retrieval-only checks close **only** via `search_policy` 's own result: BLOCKED if it returns zero candidates, PASS (with the retrieved clause as citation) if it returns any. Neither check has a computation step that applies the retrieved clause's actual text (e.g. extracting its stated day-count window and comparing against the claim's filed date) — that's deferred pending Phase 3 tagging policy chunks with structured metadata (e.g. a `window_days` field) to compute against. **This is deliberate, not an oversight:** an earlier version gave the model a separate tool that took a `window_days` argument directly from the model, intending the model to use the number

from a retrieved clause — instead the model supplied a plausible-sounding day count from its own training knowledge without ever calling `search_policy` first, closing `policy_dispute_window` on an ungrounded guess. Found via the Phase 5 end-to-end test (a `billing_dispute` claim resolved `policy_dispute_window` to PASS with `window_days: 60` despite Pinecone having 0 vectors). Removed that tool; a check able to close only through an actual retrieval hit can't be bypassed this way.

5. Open / To-Be-Decided

All previously open items now have a decision (§1, §4). Nothing outstanding at this time — revisit this section as implementation surfaces new gaps.